

Grupo 10

Informe Diseño de Compiladores

Etapas 3 y 4: Generación de Código Intermedio y Generación de Código de Máquina

Universidad Nacional del Centro de la
Provincia de Buenos Aires.

Alumnos

De Cáceres, Gonzalo

gonzadecaseres@gmail.com

Halty, Héctor Manuel

hectormanuelhalty@gmail.com

Lasarte, Fermin

fermin.lasarte@icloud.com

Tutor

José Fernández León

Temas Particulares Asignados

2 5 8 9 14 19 21 23 25 28 31 32 - b e f

Introducción

El presente informe detalla la metodología, arquitectura y decisiones de diseño adoptadas para la implementación de la etapa de Generación de Código del compilador. Este proceso abarca dos fases consecutivas: la Generación de Código Intermedio (TP3), utilizando una representación mediante Tercetos que facilita la linealización del árbol sintáctico, y la Generación de Código de Máquina (TP4), que realiza la traducción final hacia lenguaje Assembler para la arquitectura Pentium x86. El diseño contempla la implementación de los temas específicos asignados al grupo, incluyendo el manejo de tipos de datos (uint de 16 bits y float de 32 bits), estructuras de control (do-while), asignaciones múltiples, funciones con paso de parámetros por copia-resultado, expresiones lambda y controles de tiempo de ejecución (overflow en operaciones aritméticas y resta negativa). En las siguientes secciones, se detalla la lógica de implementación, las estructuras de datos utilizadas y el manejo de los desafíos encontrados.

Modificaciones a las etapas anteriores

Las principales modificaciones realizadas a los módulos léxico y sintáctico para soportar la generación de código fueron:

- **Tabla de Símbolos:** Se enriqueció la estructura para almacenar atributos complejos necesarios para la semántica, específicamente: Tipo (para inferencia y chequeos), Uso (para distinguir variables de funciones), y listas de Parámetros y TiposRetorno para soportar la sobrecarga y validaciones de invocación.
- **Gramática:** Se ajustaron las reglas de sentencia_declarativa para permitir la propagación de tipos y se integraron las acciones semánticas (Generador.java) directamente en las reglas de producción para construir los tercetos en tiempo de compilación.

Generación de Código Intermedio

La lógica central de esta etapa reside en la clase `Generador.java`. Esta clase implementa el patrón de diseño Singleton, lo que garantiza una instancia única accesible globalmente desde las acciones semánticas del parser definido en `gramatica.y`.

Información agregada a la TS

Para soportar las validaciones semánticas y la generación de código intermedio, la Tabla de Símbolos se enriqueció con atributos clave más allá del simple lexema. Se incorporó el atributo `Tipo`, determinado léxicamente para constantes (`uint`, `float`, `CadenaMultilinea`) y sintácticamente para identificadores, soportando la inferencia de tipos (Tema 9) y los retornos de función; junto con el atributo `Uso`, que distingue el rol semántico de cada entrada (`variable`, `funcion`, `parametro`, `parametro_lambda`, `nombre_programa`). Además, para cumplir con los requerimientos específicos del grupo, se añadieron atributos complejos como `RetornoMultiple` y `TiposRetorno` para gestionar funciones que devuelven múltiples valores (Tema 21), listas de Parámetros, tipos en las invocaciones, y el atributo `Pasaje` para controlar la semántica de Copia-Resultado (Tema 25), todo ello organizado mediante Name Mangling para manejar correctamente el alcance y el ocultamiento de variables en distintos ámbitos.

Notación Posicional en YACC

Para la gestión de los atributos semánticos asociados a los símbolos de la gramática, se empleó la notación posicional característica de YACC. Esta sintaxis permite acceder a los valores de los tokens y no terminales dentro de una regla de producción, facilitando la propagación de información a través del árbol de análisis sintáctico.

La notación se estructura de la siguiente manera:

- **`$$`**: Representa el valor semántico del símbolo no terminal situado en el lado izquierdo de la regla (el resultado de la reducción).
- **`$n`**: Representa el valor semántico del componente en la *n*-ésima posición del lado derecho de la regla.

En la implementación del compilador, estos valores corresponden a instancias de la clase `ParserVal`. Específicamente, se utilizó el campo `.sval` (de tipo `String`) para almacenar y recuperar los lexemas identificados. Esta mecánica fue imprescindible durante la generación de código intermedio, permitiendo recuperar las claves de la Tabla de Símbolos de los operandos para la construcción automatizada de los tercetos. El ejemplo presentado a continuación es ideal para representar el uso de la notación posicional, ya que muestra claramente cómo se toman dos valores hijos (`$1` y `$3`), se procesan (concatenación) y se asignan al padre (`$$`):

```

variable : ID PUNTO ID
        {
            $$.$sval = $1.sval + "." + $3.sval;
            $$.$ival = $1.ival;
        }
        |
        ID
        {
            $$.$sval = $1.sval;
            $$.$ival = $1.ival;
        }
        ;

```

En este fragmento, la regla gramatical define que una variable puede estar compuesta por un identificador (\$1), seguido de un punto (posición 2, ignorada semánticamente), y otro identificador (\$3). La acción semántica utiliza \$1.sval y \$3.sval para recuperar los lexemas de los identificadores y los concatena para formar el nombre cualificado (mangled), asignando el resultado a \$\$.\$sval para que esté disponible en niveles superiores de la gramática.

```

public ParserVal(int val) { ival=val; }

/**
 * Initialize me as a double
 */
public ParserVal(double val) { dval=val; }

/**
 * Initialize me as a string
 */
public ParserVal(String val) { sval=val; }

```

Como podemos observar en la imagen, el tipo de dato asociado a la variable de valor del parser, encapsulada en la estructura ParserVal, se infiere directamente a partir del identificador de la variable utilizado en su definición. Esta convención de nomenclatura es fundamental para el sistema, ya que permite determinar inequívocamente cómo debe interpretarse el valor almacenado.

Específicamente, la regla de tipificación se basa en la primera letra del nombre de la variable de miembro dentro de ParserVal:

1. **sval:** Si el nombre de la variable comienza con la letra 's', como en sval, el tipo de dato se establece como string (cadena de texto). Esto es apropiado para almacenar secuencias de caracteres, identificadores, literales de texto, o cualquier información que deba tratarse como una cadena alfanumérica.
2. **ival:** Si la variable comienza con 'i', como en ival, el tipo de dato es un entero (integer). Este tipo es ideal para representar números sin componentes fraccionarios, como contadores, índices, o valores numéricos discretos.
3. **dval:** Si la variable comienza con 'd', como en dval, el tipo de dato se interpreta como double (doble precisión). Este es un tipo de punto flotante utilizado para almacenar números reales que requieren alta precisión, esenciales en cálculos matemáticos, científicos, o financieros.

Esta técnica de codificación del tipo de dato en el nombre de la variable interna de ParserVal es una característica de diseño que simplifica la gestión de tipos dentro del parser, haciendo explícito el tipo de dato esperado para cada valor retornado por las reglas de gramática.

Tercetos

Los tercetos constituyen una estructura de representación intermedia de código organizada en tres componentes. El primer campo define la operación a realizar, mientras que el segundo y el tercero actúan como operandos, los cuales pueden ser lexemas o referencias a otros tercetos. En el caso de operaciones unarias, el tercer componente puede permanecer nulo.

Cada objeto Terceto se compone de tres atributos fundamentales:

- **Operador:** Una cadena que indica la acción a realizar (por ejemplo, +, :=, BF para bifurcaciones falsas, o CALL para invocaciones).
- **Operandos:** Dos campos (operando1 y operando2) que pueden almacenar lexemas de variables, constantes, o referencias a índices de otros tercetos previamente generados (ej. [10]).
- **Tipo:** Un atributo esencial para el chequeo semántico posterior, que almacena el tipo de dato resultante de la operación (ej. uint, float).

Visualmente el terceto tiene la siguiente forma:

$T1(+, 1, 2)$
 $T2(:=, A, [T1])$

Para el caso de un tercer componente nulo:

$T1(print, \&hola\&, -)$

```
public class Terceto { 12 usages  ⚡ Fermin Lasarte
    private String operador; 6 usages
    private String operando1; 7 usages
    private String operando2; 7 usages
    private String tipo; 5 usages

    // Constructor para operaciones binarias (ej: +, -, :=)
    public Terceto(String operador, String operando1, String operando2) {
        this.operador = operador;
        this.operando1 = operando1;
        this.operando2 = operando2;
        this.tipo = "void";
    }

    // Constructor para operaciones unarias (ej: toui, RETURN)
    public Terceto(String operador, String operando1) { 1 usage  ⚡ Fermin Lasarte
        this.operador = operador;
        this.operando1 = operando1;
        this.operando2 = null;
        this.tipo = "void";
    }

    // Constructor para saltos o etiquetas (ej: BI, LABEL)
    public Terceto(String operador) { 1 usage  ⚡ Fermin Lasarte
        this.operador = operador;
        this.operando1 = null;
        this.operando2 = null;
        this.tipo = "void";
    }

    @Override  ⚡ Fermin Lasarte
    public String toString() {
        // Muestra '_' para operandos nulos, facilitando la lectura
        String op1 = (operando1 != null) ? operando1 : "_";
        String op2 = (operando2 != null) ? operando2 : "_";
        return "(" + operador + ", " + op1 + ", " + op2 + ")";
    }
}
```

Generación de Tercetos

La generación de tercetos ocurre simultáneamente con el análisis sintáctico ascendente. Para gestionar la precedencia de operadores y el anidamiento de estructuras, se emplean pilas auxiliares dentro de la clase `Generador`:

1. **Gestión de Operandos (`pilaOperandos`):** Esta pila de tipo `Stack<String>` almacena temporalmente los lexemas o las referencias a los tercetos creados. Al reducirse una regla gramatical (como $E \rightarrow E + T$), el generador desapila los operandos necesarios, crea el nuevo terceto y apila su índice resultante para ser usado por operaciones superiores.
2. **Control de Flujo (`pilaControl`):** Utilizada primordialmente para la técnica de backpatching o "rellenado hacia atrás". Esta pila `Stack<Integer>` almacena los índices de los tercetos de salto incompleto (como BI o BT) en estructuras como IF o DO-WHILE, permitiendo completar la dirección de destino una vez que se ha procesado el bloque de código correspondiente.

```
public class Generador { 13 usages  Fermin Lasarte +1

    private static volatile Generador instance; 3 usages
    private ArrayList<Terceto> tercetos; 14 usages
    private Stack<String> pilaOperandos; 4 usages
    private Stack<Integer> pilaControl; 4 usages
    private Stack<ParametroInfo> pilaParametros; 4 usages
    private Stack<ParametroRealInfo> pilaParametrosReales; 6 usages
    private Stack<String> pilaLadoDerecho; 4 usages
    private AnalizadorLexico al; 14 usages

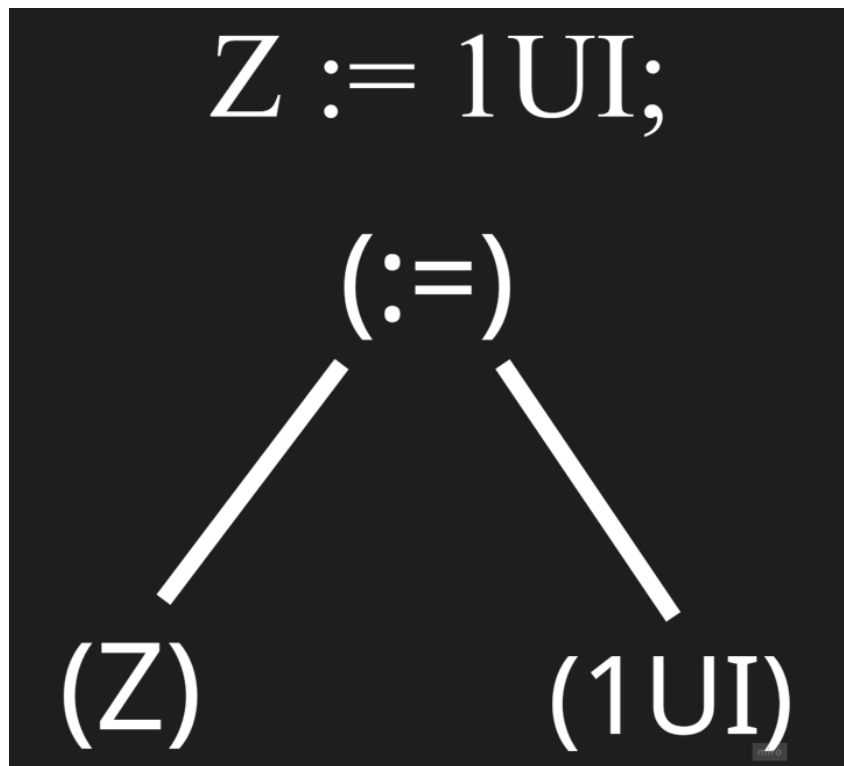
    private boolean generacionHabilitada = true; 2 usages
    private final Terceto dummyTerceto = new Terceto( operador: "DUMMY", operando1: "-", operando2: "-");

    private Generador() { 1 usage  Fermin Lasarte
        this.tercetos = new ArrayList<Terceto>();
        this.pilaOperandos = new Stack<>();
        this.pilaControl = new Stack<>();
        this.pilaParametros = new Stack<>();
        this.pilaParametrosReales = new Stack<>();
        this.pilaLadoDerecho = new Stack<>();
    }
}
```


Generación de Tercetos a partir de Árbol Sintáctico

El proceso de generación de código intermedio implica una transformación desde la estructura jerárquica implícita en la gramática hacia una representación lineal de instrucciones. Conceptualmente, esto equivale a realizar un recorrido post-orden del árbol sintáctico: se resuelven primero los nodos hijos (operandos y sub-expresiones) antes de procesar el nodo padre (operador). Dado que nuestro compilador utiliza una estrategia dirigida por la sintaxis, este "recorrido" ocurre de manera natural durante las reducciones ascendentes del parser, garantizando que cuando se alcanza una regla de producción (como una asignación o una suma), los operandos necesarios ya han sido procesados y sus referencias están disponibles en la pila de control.

Para ilustrar esta lógica, consideremos la sentencia de asignación simple $Z := 1UI$; Estructuralmente, esta instrucción se puede representar mediante el siguiente árbol sintáctico, donde la raíz denota la operación y las hojas los operandos:



Durante el análisis, el compilador procesa primero las hojas del árbol. Al identificar la constante 1UI (lado derecho) y la variable Z (lado izquierdo), se valida su información en la Tabla de Símbolos. Posteriormente, la reducción gramatical asciende hacia el nodo raíz que representa la operación de asignación $:=$. En este momento exacto, el Generador captura los operandos resueltos y emite el terceto correspondiente, aplanando la jerarquía en una instrucción secuencial atómica: $(:=, Z, 1UI)$. Esta linealización es fundamental, ya que descompone expresiones complejas en pasos discretos que facilitan la posterior traducción directa a instrucciones de código Assembler.

Temas Particulares

Tema 2 - Enteros sin signo - uint: En la etapa léxica se valida el sufijo UI y el rango (0-65535). En la generación de código, se utilizan registros de 32 bits (EAX) pero se incorpora un chequeo explícito de rango tras cada operación aritmética para asegurar que el resultado no exceda los 16 bits permitidos.

Tema 5 (Punto Flotante - float): Se valida la notación científica y los rangos de 32 bits en el léxico. En Assembler, todas las operaciones utilizan la pila del coprocesador matemático (FPU) mediante instrucciones como FLD (cargar), FADD (sumar) y FSTP (guardar).

Tema 8 (Cadenas Multilínea): Se reconocen léxicamente y se almacenan en la Tabla de Símbolos. En la generación de Assembler, se declaran en la sección .data como bytes terminados en nulo (db . . . , 0), manejando los saltos de línea internos.

Tema 9 (Inferencia - var): La resolución de este tema es puramente semántica. El tipo se infiere y registra en la Tabla de Símbolos durante el análisis sintáctico. Para la generación de código, el tratamiento es transparente ya que el Generador recupera el tipo ya resuelto de la variable directamente de la Tabla de Símbolos.

Tema 14 - Sentencia do-while: Para la estructura iterativa do-while, el compilador marca el inicio del bucle apilando el índice del siguiente terceto libre en pilaControl antes de procesar el cuerpo. Tras generar el código del bloque, se evalúa la condición mediante un terceto de comparación. Finalmente, se emite un terceto BT (Branch True) que, en caso de cumplirse la condición, realiza un salto hacia el índice recuperado de la pila, cerrando así el ciclo.

Tema 31 (Conversión explícita - toui): Se genera un terceto unario (TOUI, operando, _). En la etapa de Assembler, esto se traduce utilizando la instrucción FISTP (Floating Point Integer Store), que convierte el valor en el tope de la pila flotante a un entero y lo almacena en la variable de destino.

Para formalizar la lógica descrita, el algoritmo utilizado para la generación de las bifurcaciones en la sentencia do-while sigue el siguiente pseudocódigo:

Pseudocódigo Generación Do-While

1. Al encontrar el token do:
 - Se obtiene el índice del próximo terceto disponible (inicioBucle).
 - Se apila este índice en la pilaControl.
2. Se procesa el bloque de sentencias ejecutables.
3. Al encontrar el token while y la (condicion):
 - Se genera el código intermedio para evaluar la condición.
 - Se desapila el operando con el resultado de la condición (cond).
 - Se desapila el destino del salto de la pilaControl (destino).
 - Se genera un terceto de bifurcación verdadera: BT, cond, destino.

```

condicional_do_while: DO
{
    g.apilarControl(g.getProximoTerceto());
}
bloque_ejecutable WHILE '(' condicion ')' ':'
{
    Object lineaObj = al.getAtributo("do", "Linea");
    salida.add("Linea " + $7.ival + ": Sentencia DO-WHILE reconocida.");
    String refCondicion = g.desapilarOperando();
    int inicioBucle = g.desapilarControl();
    if (refCondicion.equals("ERROR_CONDICION")) {
        al.agregarErrorSemantico("Linea " + $7.ival + ": Error Semantico: No se genero el salto del DO-WHILE debido a una condicion invalida.");
    } else {
        String tercetoSalto = g.addTerceto("BT", refCondicion, "[" + inicioBucle + "]");
    }
}
;

```

La lógica de control implementada respeta la naturaleza de post-condición del do-while. La teoría dicta que el cuerpo del bucle debe ejecutarse al menos una vez (fall-through), por lo que el salto condicional (BT) se coloca al final y solo debe ocurrir si la condición se cumple, invirtiendo la lógica tradicional de los bucles while (pre-condición).

Temas 19 y 21 – Asignaciones y Retornos Múltiples: Dado que el lenguaje permite asignaciones múltiples (Tema 19) y funciones que retornan múltiples valores (Tema 21), se implementó una pila adicional llamada pilaLadoDerecho. Esta estructura acumula las expresiones o invocaciones a funciones. En el caso de funciones con múltiples retornos, se genera un terceto especial denominado GET_RET. Este operador se encarga de extraer un valor específico devuelto por la función, utilizando la sintaxis (GET_RET, [terceto_call], indice_retorno). El sistema valida rigurosamente que la cantidad de variables a la izquierda sea compatible con los retornos disponibles. Siguiendo la especificación del Tema 21, si el número de retornos excede a las variables receptoras, se descartan los sobrantes y se emite el Warning correspondiente.

```

if (cantRetornos > cantIzquierda) {
    al.agregarWarning("Linea " + lineaActual + ": Warning (Tema 21): Funcion '" + funcName + "' retorna " + cantRetornos + " valores, pero solo se asignan " + c);
}
for (int i = 0; i < cantIzquierda; i++) {
    String var = listaVariables.get(i);
    String tipoVar = g.getTipo(var);
    String tipoRet = tiposRetorno.get(i);
    if (g.chequearAsignacion(tipoVar, tipoRet, Integer.parseInt(lineaActual))) {
        String retTerceto = g.addTerceto("GET_RET", funcTerceto, String.valueOf(i));
        g.getTerceto(Integer.parseInt(retTerceto.substring(1, retTerceto.length()-1))).setTipo(tipoRet);
        g.addTerceto(":", var, retTerceto);
    }
}

```

Tema 28 - Expresiones Lambda: El manejo de expresiones Lambda pasadas como parámetros presentó un desafío de alcance y flujo. Al detectar una definición lambda, el generador inserta un salto incondicional (BI) para evitar que el cuerpo de la función anónima se ejecute durante la definición. Se genera una etiqueta única y un nuevo ámbito (ej. lambda_170) para encapsular sus variables. El terceto resultante de esta definición no ejecuta la función, sino que retorna una referencia a su etiqueta de inicio. Esta referencia se pasa a la función receptora como si fuera un puntero a función. Dentro del cuerpo de la lambda, se utiliza el terceto RET_LAMBDA para gestionar el retorno de control al finalizar su ejecución.

```
lambda_expresion : (' tipo ID ')' '{'
{
    pilaSaltosLambda.push(g.addTerceto("BI", "_", "_"));
    int inicioLambda = g.getProximoTerceto();
    $$sval = "L" + String.valueOf(inicioLambda);
    g.abrirAmbito("lambda_" + inicioLambda);
    al.agregarLexemaTS($$.sval);
    al.agregarAtributoLexema($$.sval, "Uso", "parametro_lambda");
    al.agregarAtributoLexema($$.sval, "Tipo", $2.sval);
    g.addTerceto("DEF_PARAM", $$.sval, "_");
    $$ival = $1.ival;
}
cuerpo_lambda '}';
{
    g.addTerceto("RET_LAMBDA", "_", "_");
    int tercetoFin = g.getProximoTerceto();
    String saltoIncondicional = pilaSaltosLambda.pop();
    g.modificarSaltoTerceto(Integer.parseInt(saltoIncondicional.substring(1, saltoIncondicional.length()-1)), "[" + tercetoFin + "]");
    g.cerrarAmbito();
    $$sval = $6.sval;
    $$ival = $1.ival;
}
;
```

Tema 23 - Prefijado Opcional: Para resolver la visibilidad de variables con prefijado opcional, se implementó la técnica de Name Mangling desde el AnalizadorLexico. Las variables se almacenan en la Tabla de Símbolos concatenando su lexema con su ámbito. El Generador intenta resolver las referencias buscando primero en el ámbito local; de no encontrarse, escala jerárquicamente hacia los ámbitos padres, o utiliza el prefijo explícito (ej. MAIN.A) si este fue provisto por el programador.

```
public String getNombreMangled(String lexema) {
    if (lexema.contains(".")) {
        String[] parts = lexema.split("\\.");
        String scopeName = parts[0];
        String varName = parts[1];
        for (Pair<String, HashMap<String, HashMap<String, Object>>> p : tablaSimbolos) {
            String ambito = p.getKey();
            if (ambito.equals(scopeName) || ambito.endsWith(":" + scopeName)) {
                String candidate = varName + ":" + ambito;
                if (p.getValue().containsKey(candidate)) {
                    return candidate;
                }
            }
        }
        return lexema;
    }
    int idx = indiceAmbitoActual;
    while (idx != -1) {
        Pair<String, HashMap<String, HashMap<String, Object>>> scope = tablaSimbolos.get(idx);
        String ambitoNombre = scope.getKey();
        String lexemaMangled = lexema + ":" + ambitoNombre;
        HashMap<String, HashMap<String, Object>> ambitoMap = scope.getValue();
        if (ambitoMap.containsKey(lexemaMangled)) {
            return lexemaMangled;
        }
        if (ambitoMap.containsKey("...METADATA...") && ambitoMap.get("...METADATA...").containsKey("PARENT")) {
            idx = (Integer) ambitoMap.get("...METADATA...").get("PARENT");
        } else {
            idx = -1;
        }
    }
    return lexema;
}
```

Tema 25 - Semántica de Pasaje Copia-Resultado: En cumplimiento con el Tema 25, se implementó la semántica de "Copia-Resultado". Esto implica que, al finalizar la ejecución de una función, los valores finales de los parámetros formales deben copiarse de vuelta a los parámetros reales (siempre que sean variables modificables). Esto se gestiona en el código intermedio asegurando que las modificaciones dentro de la función se realicen sobre variables locales y, previo al retorno, se actualicen las referencias externas correspondientes.

A diferencia del pasaje por referencia (que opera sobre direcciones de memoria compartidas), la semántica de copia-resultado aísla la ejecución de la función. Los efectos colaterales sobre los parámetros reales solo se materializan atómicamente si la función termina exitosamente, evitando estados inconsistentes en las variables externas durante la ejecución intermedia del llamado.

```
String terceto = g.addTerceto("CALL", funcName);
g.getTerceto(Integer.parseInt(terceto.substring(1, terceto.length()-1))).setTipo(al.getAtributo(funcName, "Tipo").toString());
$$sval = terceto;

for (ParametroRealInfo real : reales) {
    if (real.nombreFormal != null) {
        ParametroInfo formal = formales.stream().filter(f -> f.nombre.equals(real.nombreFormal)).findFirst().orElse(null);
        if (formal != null && (formal.pasaje.equals("cr_se") || formal.pasaje.equals("cr_le"))) {
            String mangledFunc = al.getNombreMangled(funcName);
            String nombreParametro = formal.nombre;
            if (mangledFunc.contains(":")) {
                int firstColon = mangledFunc.indexOf(':');
                String name = mangledFunc.substring(0, firstColon);
                String scope = mangledFunc.substring(firstColon + 1);
                String scopeBody = scope + ":" + name;
                nombreParametro = formal.nombre + ":" + scopeBody;
            }
            g.addTerceto(":= ", real.operando, nombreParametro);
        }
    }
}
```

Generación de Código Assembler

Una vez obtenidos los tercetos del código intermedio, el sistema avanza hacia la emisión del código assembler compatible con MASM32. La ejecución de este archivo de salida permite realizar las pruebas funcionales dinámicas necesarias para verificar el cumplimiento de las reglas semánticas y de control de flujo.

Mecanismo General

Se utilizó el método de Variables Auxiliares. Para cada terceto que produce un resultado numérico, se reserva una variable de memoria (@auxN).

- **Enteros (uint):** Se utilizan los registros de propósito general de 32 bits (EAX, EDX).
- **Flotantes (float):** Se utiliza la pila del coprocesador matemático 80x87 (FLD, FSTP, FADD, etc.).

Mecanismo de generación de etiquetas de salto

Para resolver las bifurcaciones (saltos condicionales e incondicionales), se utilizó un esquema de etiquetado basado en los índices de los tercetos. Cada terceto generado tiene asociado una etiqueta potencial en el código Assembler.

Las etiquetas se construyen concatenando el prefijo fijo "Label" con el número de índice del terceto destino. Por ejemplo, si un terceto (BI, ↓, [15]) indica un salto al terceto número 15, el generador producirá la instrucción JMP Label15. Antes de generar el código para el terceto 15, se inserta la etiqueta Label15: en el archivo de salida.

Estructura del Archivo Salida

El archivo generado consta de:

1. **Header:** Definiciones de arquitectura (.386, .model flat, stdcall) e includes de librerías (msvcrt, kernel32).
2. **.data:** Declaración de todas las variables del usuario (con mangling), variables auxiliares (@aux), cadenas de texto para PRINT y constantes para control de errores.
3. **.code:** Etiqueta start: seguida de la traducción secuencial de tercetos.

Traducción de Operaciones Aritméticas

El compilador distingue dinámicamente si la operación es flotante o entera consultando el tipo del tercer o de sus operandos.

- **Suma Entera (+):**

```
MOV EAX, op1
ADD EAX, op2
MOV @auxN, EAX
```

- **Suma Flotante:**

```
FLD op1
FLD op2
FADD
FSTP @auxN
```

Manejo de Comparaciones

Para las comparaciones (<, >, ==), se utilizó el registro de estado y las instrucciones SETxx. Por ejemplo, para < (menor): Se realiza CMP (enteros) o FCOMIP (flotantes), y luego SETB AL (Set if Below), moviendo el resultado (0 o 1) a la variable auxiliar.

Controles en Tiempo de Ejecución

Siguiendo las consignas del Trabajo Práctico N°4, se incorporaron chequeos automáticos en el código Assembler generado para asegurar la integridad de la ejecución. Si se detecta un error, el programa emite un mensaje y finaliza invocando ExitProcess.

Los controles asignados al grupo (b, e, f) se resolvieron de la siguiente manera:

b) Overflow en Sumas de Enteros

El tipo uint tiene un límite de 16 bits (65535), pero se opera en registros de 32 bits. Se verifica que el resultado no exceda este límite explícitamente.

- **Implementación:** Después de un ADD EAX, op2, se compara EAX con 65535.

```
codigo.append("ADD EAX, ").append(op2).append("\n");
codigo.append("CMP EAX, 65535\n");
codigo.append("JA ErrorOverflow\n");
```

e) Overflow en Productos de Datos de Punto Flotante

Para las operaciones flotantes, se verifica que el valor absoluto del resultado no exceda el máximo valor representable definido en la consigna.

- **Implementación:** Se definió `MaxFloatValue` dd 2139095039 en la sección de datos. Tras una operación (ej. `FMUL`), se compara el resultado.

```
codigo.append("FMUL\n");  
codigo.append("FLD ST(0)\n");  
codigo.append("FABS\n");  
codigo.append("FCOMP MaxFloatValue\n");  
codigo.append("FSTSW AX\n");  
codigo.append("SAHF\n");  
codigo.append("JA ErrorOverflow\n");
```

f) Resultados Negativos en Restas de Enteros sin Signo

Dado que el tipo `uint` no admite valores negativos, una resta $A - B$ donde $B > A$ debe producir un error en tiempo de ejecución (`underflow`).

- **Implementación:** Se utiliza el flag de acarreo (`Carry Flag`) que se activa si la resta requiere un "préstamo" (lo que indica resultado negativo en aritmética sin signo).

```
codigo.append("SUB EAX, ").append(op2).append("\n");  
codigo.append("JC ErrorRestaNegativa\n");
```


Almacenamiento y Gestión de Variables Auxiliares

Las variables auxiliares se generan dinámicamente durante la creación de los tercetos y se almacenan directamente en la Tabla de Símbolos.

Como se puede observar en el archivo “*salida_parser.txt*”, las variables auxiliares aparecen listadas junto con las variables del usuario, identificadas con el lexema @auxN (donde N es el índice del terceto) y el atributo Uso: variable_auxiliar.

Siguiendo los principios de diseño de compiladores, las variables temporales o auxiliares generadas para la representación de código intermedio deben ser gestionadas formalmente por la Tabla de Símbolos por las siguientes razones:

- **Gestión de Memoria Unificada:** El generador de código final necesita reservar espacio de memoria estática para todas las entidades que almacenan valores, no solo las declaradas por el usuario. Al incluirlas en la Tabla de Símbolos, el compilador puede iterar sobre una única estructura para generar las directivas de datos de manera uniforme.
- **Resolución de Direcciones y Alcance:** Al igual que las variables de usuario, las auxiliares deben poseer un ámbito para evitar conflictos de nombres y asegurar que se accede a la dirección de memoria correcta durante la ejecución. Esto se evidencia mediante el Name Mangling aplicado también a las auxiliares (ej. @aux189:MAIN).
- **Consistencia de Atributos:** Permite asociar metadatos esenciales, como el tipo de dato que almacenan temporalmente, facilitando comprobaciones o conversiones posteriores sin requerir estructuras de datos paralelas y redundantes.

Consideraciones Generales

En respuesta a la devolución recibida sobre la entrega de las Etapas 3 y 4, y con el objetivo de clarificar el estado actual del compilador, se detallan a continuación las resoluciones a los puntos específicos señalados.

Clarificación sobre la Sintaxis en Salida y Casos de Prueba

Respecto a la notación utilizada en los archivos de prueba y en la impresión de tercetos (salida_parser.txt):

- **Símbolo @:** Corresponde al Tema 32 (Comentarios de una línea). En el código fuente, todo lo que sigue a un @ es ignorado por el lexer hasta el salto de línea. En los archivos de salida, se utiliza visualmente para indicar metadatos de depuración (como @Numero de línea:), pero esto es producto de los print del compilador, no parte de la sintaxis ejecutable.
- **Símbolo &:** Corresponde al delimitador de cadenas multilínea (Tema 8). Los strings en nuestro lenguaje se encierran entre ampersands (ej. &Hola Mundo&).
- **Símbolo _ en Tercetos:** En la representación intermedia (Tercetos), el guión bajo _ se utiliza como placeholder para indicar un operando nulo o vacío (por ejemplo, en saltos incondicionales (BI, _, [10]) o en operaciones unarias).

Decisión de Diseño en Generación de Código Assembler

Respecto a la eficiencia del código generado, el equipo es consciente de que la estrategia actual, basada en el uso de variables auxiliares en memoria para cada operación intermedia, conlleva una sobrecarga de accesos a la RAM en comparación con técnicas de asignación de registros (*Register Allocation*). Aunque reconocemos que maximizar la permanencia de valores en los registros de la CPU optimizaría el tiempo de ejecución, ante la ausencia de especificaciones estrictas sobre optimización en la consigna, se priorizó la robustez y la corrección de la traducción. Esta decisión de diseño garantiza la integridad del flujo de datos y simplifica la depuración, cumpliendo con los objetivos académicos de la etapa sin introducir la complejidad de algoritmos avanzados de gestión de registros.

Conclusión

La finalización de las etapas de Generación de Código Intermedio (TP3) y Generación de Código Assembler (TP4) culmina exitosamente el desarrollo del compilador, logrando transformar código fuente de alto nivel en un ejecutable funcional. La asignación de una estructura de Tercetos resultó fundamental, proporcionando una representación intermedia flexible que facilitó la resolución de construcciones complejas como las expresiones Lambda y la gestión de ámbitos mediante Name Mangling.

El proceso de traducción a Assembler permitió consolidar el manejo de tipos, integrando eficazmente el uso de registros generales para enteros sin signo y el coprocesador matemático para punto flotante. Asimismo, la implementación de controles estrictos en tiempo de ejecución para desbordamientos y operaciones inválidas garantiza la robustez del código generado. Este trabajo representa un cierre sólido del proyecto, demostrando no solo el cumplimiento de los objetivos funcionales, sino también la aplicación práctica de los conceptos teóricos adquiridos durante la cursada.

La finalización de estas etapas marca el logro de un compilador funcional y sólido, capaz de procesar correctamente el lenguaje definido y generar código ejecutable. Durante todo el desarrollo, se priorizó un enfoque de trabajo ordenado y escalable, integrando los conocimientos adquiridos en la cursada 2025 para superar los desafíos técnicos presentados. Este cierre exitoso no solo refleja el esfuerzo y la dedicación invertidos, sino que demuestra el dominio de los conceptos fundamentales necesarios para la construcción de un compilador robusto.